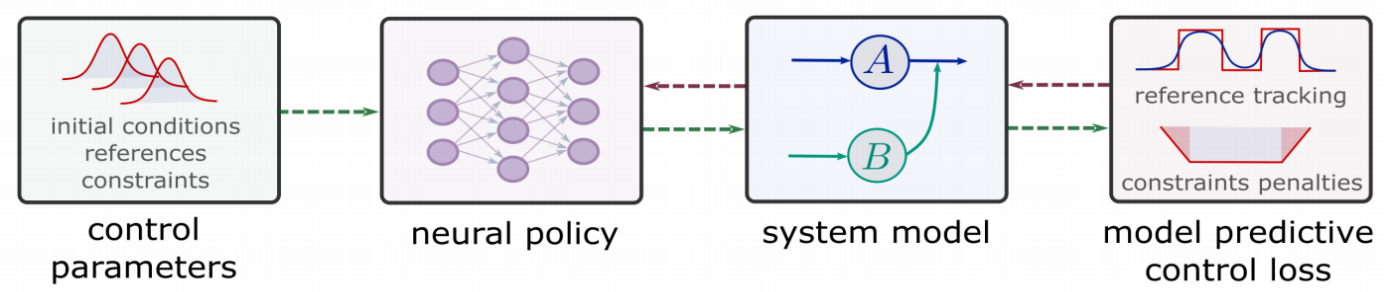


SciML Tutorial: Learning to Control (L2C)

Ján Drgoňa
Associate Professor
Civil and Systems Engineering Department
Electrical and Computer Engineering Department (secondary)
The Ralph O'Connor Sustainable Energy Institute (ROSEI)
Data Science and AI Institute (DSAI)
Johns Hopkins University

8th Annual Learning for Dynamics & Control Conference
University of Southern California, June 17

Learning to Control (L2C) Problem Formulation

Policy Learning for Parametric Optimal Control

Consider the finite horizon parametric optimal control problem

$$\begin{aligned} \min_{\{u_{0:N-1}\}} \quad & J(x_0, \xi) = \sum_{k=0}^{N-1} \ell(x_k, u_k; \xi) + \ell_N(x_N; \xi) \\ \text{s.t.} \quad & x_{k+1} = f(x_k, u_k; \xi), \\ & g(x_k, u_k; \xi) \leq 0, \\ & h(x_k, u_k; \xi) = 0. \end{aligned}$$

The optimal control solution operator is $\Pi^\star : (x_k, \xi) \mapsto u_k^\star(x_k; \xi)$.
L2C aims to learn a parameterized policy $\pi_\theta : (x_k, \xi) \mapsto u_k$, that approximates $\Pi^\star$ across operating conditions $x_0 \sim \mathcal{X}_0$ and $\xi \sim \mathcal{D}$.

Learning to Control (L2C) Problem Formulation

Policy Learning for Parametric Optimal Control

Consider the finite horizon parametric optimal control problem

$$\min_{\{u_{0:N-1}\}} \quad J(x_0, \xi) = \sum_{k=0}^{N-1} \ell(x_k, u_k; \xi) + \ell_N(x_N; \xi)$$

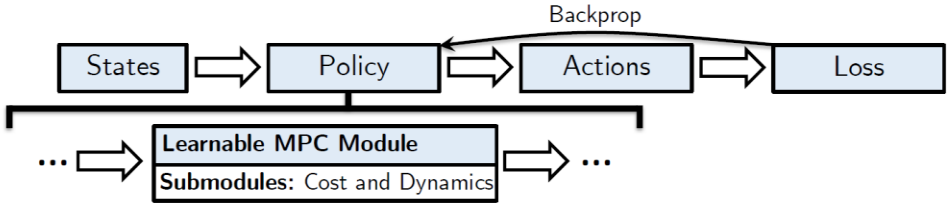
$$\text{s.t.} \quad x_{k+1} = f(x_k, u_k; \xi),$$
$$g(x_k, u_k; \xi) \leq 0,$$
$$h(x_k, u_k; \xi) = 0.$$

Assume we have differentiable cost function terms, system dynamics, constraints, and known distributions of scenarios.

The optimal control solution operator is $\Pi^\star : (x_k, \xi) \mapsto u_k^\star(x_k; \xi)$.
L2C aims to learn a parameterized policy $\pi_\theta : (x_k, \xi) \mapsto u_k$, that approximates $\Pi^\star$ across operating conditions $x_0 \sim \mathcal{X}_0$ and $\xi \sim \mathcal{D}$.

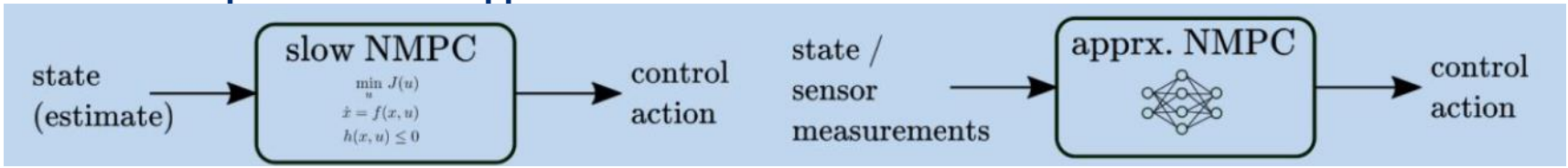
Model-Based Learning to Control (L2C) Methodologies

Supervised L2C: Differentiable Model Predictive Control



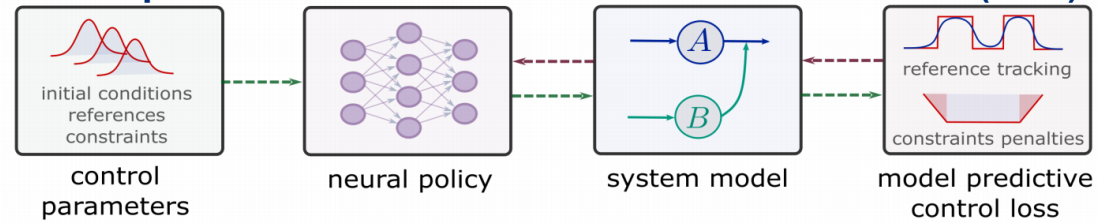
Brandon Amos, Ivan Dario Jimenez Rodriguez, Jacob Sacks, Byron Boots, and J. Zico Kolter, Differentiable MPC for End-to-end Planning and Control, NeurIPS 2018.

Supervised L2C: Approximate Model Predictive Control



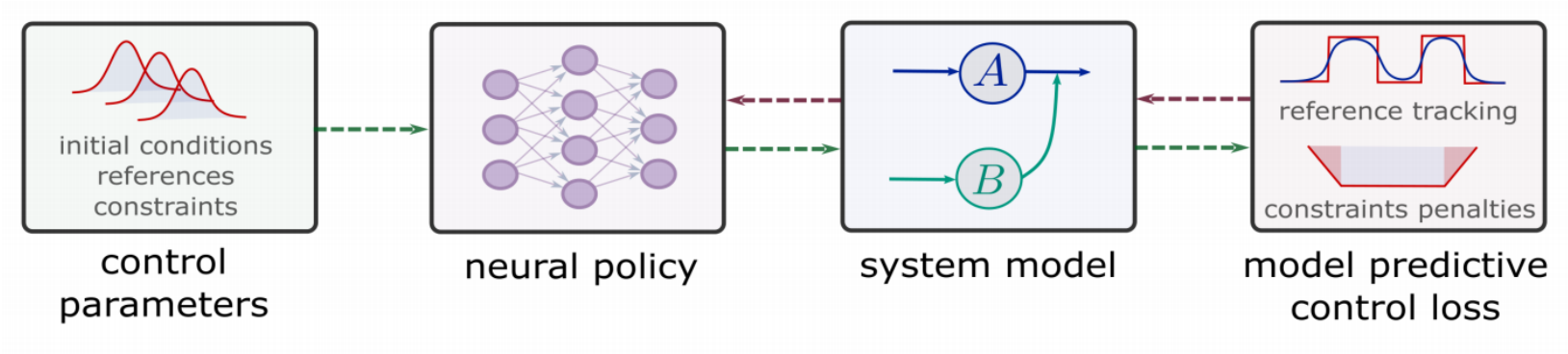
M. Hertneck, et al., Learning an Approximate Model Predictive Controller With Guarantees, in IEEE Control Systems Letters, 2018
B. Karg and S. Lucia, Efficient Representation and Approximation of Model Predictive Control Laws via Deep Learning, in IEEE Transactions on Cybernetics, 2020
S Chen, et al. Approximating explicit model predictive control using constrained neural networks, ACC, 2018

Self-Supervised L2C: Differentiable Predictive Control (DPC)

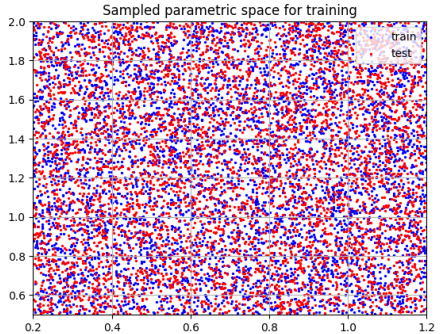


J. Drgoňa, A. Tuor and D. Vrabie, Learning Constrained Parametric Differentiable Predictive Control Policies With Guarantees, in IEEE TSMC, 2024

Self-Supervised L2C: Differentiable Predictive Control (DPC)



Dataset: collocation points in the control parametric space.



Architecture: differentiable model with neural network control policy.

$$x_{k+1} = \text{ODESolve}(f(x_k, u_k))$$
$$u_k = NN_\theta(x_k, \xi_k)$$
$$g(x_k, u_k, \xi_k) \leq 0$$
$$x_0 \sim \mathcal{P}_{x_0}$$
$$\xi_k \sim \mathcal{P}_\xi$$

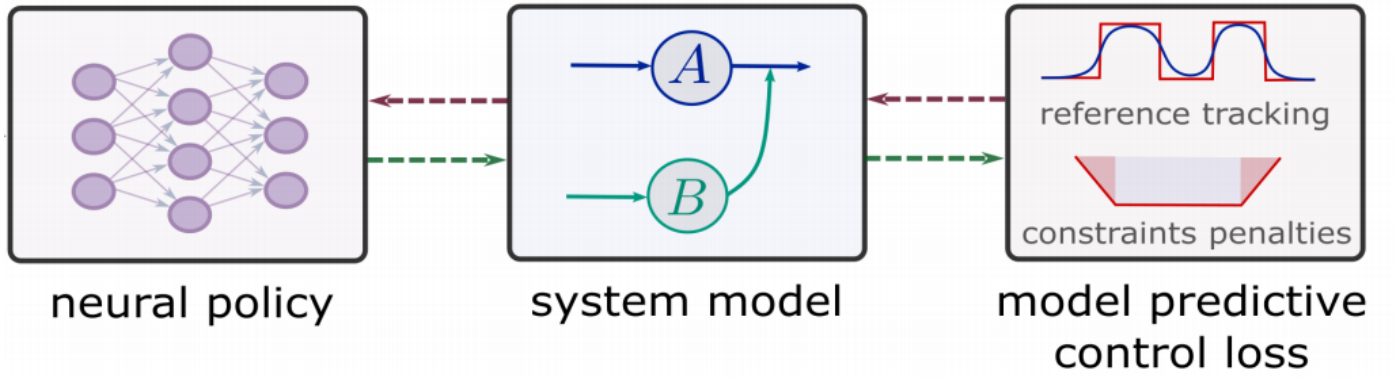
Loss function: reference tracking, constraints and terminal penalties.

$$\ell_1 = \sum_{i=1}^{m} \sum_{k=1}^{N-1} Q_x ||x_k^i - r_k^i||_2^2$$
$$\ell_2 = \sum_{i=1}^{m} \sum_{k=1}^{N-1} Q_g ||\texttt{RELU}(g(x_k^i, u_k^i, \xi_k^i)||_2^2$$
$$\ell_{L2C} = \ell_1 + \ell_2$$

https://github.com/pnnl/neuromancer/blob/master/examples/control/Part_3_ref_tracking_ODE.ipynb

J. Drgoňa, A. Tuor and D. Vrabie, "Learning Constrained Parametric Differentiable Predictive Control Policies With Guarantees," in IEEE Transactions on Systems, Man, and Cybernetics: Systems, 2024

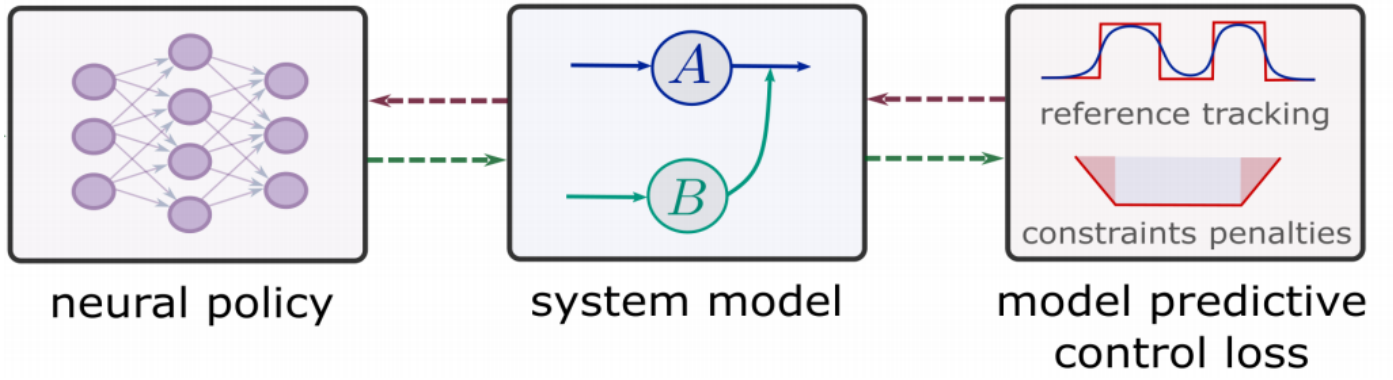
Differentiable Closed-Loop System



Forward pass. For a single scenario, the closed-loop recursion and loss are

$$
\begin{aligned}
u_k &= \pi_\theta(x_k;\xi_k),\\
x_{k+1} &= f(x_k,u_k;\xi_k),\\
\ell_k &= \ell(x_k,u_k;\xi_k) \;+\; p_x\big(g(x_k;\xi_k)\big) \;+\; p_u\big(h(u_k;\xi_k)\big),\\
\mathcal{L} &= \frac{1}{N}\sum_{k=0}^{N-1}\ell_k \;+\; \ell_N(x_N;\xi_N).
\end{aligned}
$$

Differentiable Closed-Loop System



Forward pass. For a single scenario, the closed-loop recursion and loss are

$$
\begin{aligned}
u_k &= \pi_\theta(x_k;\xi_k),\\
x_{k+1} &= f(x_k,u_k;\xi_k),\\
\ell_k &= \ell(x_k,u_k;\xi_k) \;+\; p_x\big(g(x_k;\xi_k)\big) \;+\; p_u\big(h(u_k;\xi_k)\big),\\
\mathcal{L} &= \frac{1}{N}\sum_{k=0}^{N-1}\ell_k \;+\; \ell_N(x_N;\xi_N).
\end{aligned}
$$

From an RL perspective, the DPC loss can be seen as a white-box value function suitable for gradient-based policy optimization.

Differentiable Closed-Loop System

Backward pass (sensitivities). Let

$$A_k := \frac{\partial f}{\partial x}, \quad B_k := \frac{\partial f}{\partial u}, \quad P_k := \frac{\partial \pi_\theta}{\partial x}, \quad G_k := \frac{\partial \pi_\theta}{\partial \theta} \quad \text{(all evaluated at } (x_k, u_k; \xi_k)).$$

Define $s_N := \frac{\partial \ell_N}{\partial x}(x_N; \xi_N)$ and stage gradients

$$\ell_{x,k} := \frac{\partial \ell}{\partial x} + \frac{\partial p_x}{\partial x}, \qquad \ell_{u,k} := \frac{\partial \ell}{\partial u} + \frac{\partial p_u}{\partial u}.$$

Then the reverse recursion for $k = N-1, \ldots, 0$ is

$$s_k \;=\; \ell_{x,k} \;+\; A_k^\top s_{k+1} \;+\; P_k^\top \big(\ell_{u,k} + B_k^\top s_{k+1}\big),$$

and the gradient w.r.t. the policy parameters is

$$\nabla_\theta \mathcal{L} \;=\; \sum_{k=0}^{N-1} G_k^\top \big(\ell_{u,k} + B_k^\top s_{k+1}\big).$$

Differentiable Closed-Loop System

Backward pass (sensitivities). Let

$$A_k := \frac{\partial f}{\partial x}, \quad B_k := \frac{\partial f}{\partial u}, \quad P_k := \frac{\partial \pi_\theta}{\partial x}, \quad G_k := \frac{\partial \pi_\theta}{\partial \theta} \quad \text{(all evaluated at } (x_k, u_k; \xi_k)).$$

Define $s_N := \frac{\partial \ell_N}{\partial x}(x_N; \xi_N)$ and stage gradients

$$\ell_{x,k} := \frac{\partial \ell}{\partial x} + \frac{\partial p_x}{\partial x}, \qquad \ell_{u,k} := \frac{\partial \ell}{\partial u} + \frac{\partial p_u}{\partial u}.$$

Then the reverse recursion for $k = N-1, \ldots, 0$ is

$$s_k \;=\; \ell_{x,k} \;+\; A_k^\top s_{k+1} \;+\; P_k^\top \big(\ell_{u,k} + B_k^\top s_{k+1}\big),$$

and the gradient w.r.t. the policy parameters is

$$\nabla_\theta \mathcal{L} \;=\; \sum_{k=0}^{N-1} G_k^\top \big(\ell_{u,k} + B_k^\top s_{k+1}\big).$$

The backward pass in DPC is mathematically identical to **Backpropagation Through Time (BPTT)** used to train recurrent neural networks.

Differentiable Closed-Loop System

Backward pass (sensitivities). Let

$$A_k := \frac{\partial f}{\partial x}, \quad B_k := \frac{\partial f}{\partial u}, \quad P_k := \frac{\partial \pi_\theta}{\partial x}, \quad G_k := \frac{\partial \pi_\theta}{\partial \theta} \quad \text{(all evaluated at } (x_k, u_k; \xi_k)).$$

Define $s_N := \frac{\partial \ell_N}{\partial x}(x_N; \xi_N)$ and stage gradients

$$\ell_{x,k} := \frac{\partial \ell}{\partial x} + \frac{\partial p_x}{\partial x}, \qquad \ell_{u,k} := \frac{\partial \ell}{\partial u} + \frac{\partial p_u}{\partial u}.$$

Then the reverse recursion for $k = N-1, \ldots, 0$ is

$$s_k \;=\; \ell_{x,k} \;+\; A_k^\top s_{k+1} \;+\; P_k^\top \big(\ell_{u,k} + B_k^\top s_{k+1}\big),$$

and the gradient w.r.t. the policy parameters is

$$\nabla_\theta \mathcal{L} \;=\; \sum_{k=0}^{N-1} G_k^\top \big(\ell_{u,k} + B_k^\top s_{k+1}\big).$$

The backward pass in DPC computes discrete adjoint sensitivities **leading to Pontryagin adjoint equations** in continuous time limit**.**

DPC Policy Optimization Algorithm

Algorithm 5: Self-Supervised DPC Policy Optimization

Input: Horizon N, batch size B; initial-state distribution $\mathcal{X}_0$, scenario distribution $\mathcal{D}$; dynamics $x_{k+1} = f(x_k, u_k; \xi_k)$; policy class $u_k = \pi_\theta(x_k; \xi_k)$; costs ℓ, ℓ_N; penalties p_x, p_u with weights Q_x, Q_u; constraints g, h; optimizer $\mathbb{O}$

Output: Trained explicit policy $\pi_{\theta^\star}$

while *not converged* **do**

  Sample mini-batch $\{(x_0^{(i)}, \xi^{(i)})\}_{i=1}^{B} \sim \mathcal{X}_0 \times \mathcal{D}$;

  for $i \leftarrow 1$ **to** B **do**

    for $k \leftarrow 0$ **to** $N-1$ **do**

      $u_k^{(i)} \leftarrow \pi_\theta\big(x_k^{(i)}; \xi_k^{(i)}\big)$;

      $x_{k+1}^{(i)} \leftarrow f\big(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}\big)$;

      $\ell_k^{(i)} \leftarrow \ell\big(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}\big) + Q_x\, p_x\big(g(x_k^{(i)}; \xi_k^{(i)})\big) + Q_u\, p_u\big(h(u_k^{(i)}; \xi_k^{(i)})\big)$;

    $\mathcal{L}^{(i)} \leftarrow \frac{1}{N} \sum_{k=0}^{N-1} \ell_k^{(i)} + \ell_N\big(x_N^{(i)}; \xi_N^{(i)}\big)$;

  $\mathcal{L} \leftarrow \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}^{(i)}$; // batch DPC loss

  Compute $\nabla_\theta \mathcal{L}$ e.g., BPTT or adjoint sensitivity

  $\theta \leftarrow \mathbb{O}\big(\theta, \nabla_\theta \mathcal{L}\big)$; // e.g., Adam step

return π_θ;

DPC Policy Optimization Algorithm

Algorithm 5: Self-Supervised DPC Policy Optimization

Input: Horizon N, batch size B; initial-state distribution $\mathcal{X}_0$, scenario distribution $\mathcal{D}$; dynamics $x_{k+1} = f(x_k, u_k; \xi_k)$; policy class $u_k = \pi_\theta(x_k; \xi_k)$; costs ℓ, ℓ_N; penalties p_x, p_u with weights Q_x, Q_u; constraints g, h; optimizer $\mathbb{O}$

Output: Trained explicit policy $\pi_{\theta^\star}$

while *not converged* **do**

  Sample mini-batch $\{(x_0^{(i)}, \xi^{(i)})\}_{i=1}^{B} \sim \mathcal{X}_0 \times \mathcal{D}$;

  for $i \leftarrow 1$ **to** B **do**

    for $k \leftarrow 0$ **to** $N-1$ **do**

      $u_k^{(i)} \leftarrow \pi_\theta\big(x_k^{(i)}; \xi_k^{(i)}\big)$;

      $x_{k+1}^{(i)} \leftarrow f\big(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}\big)$;

      $\ell_k^{(i)} \leftarrow \ell\big(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}\big) + Q_x\, p_x\big(g(x_k^{(i)}; \xi_k^{(i)})\big) + Q_u\, p_u\big(h(u_k^{(i)}; \xi_k^{(i)})\big)$;

    $\mathcal{L}^{(i)} \leftarrow \frac{1}{N} \sum_{k=0}^{N-1} \ell_k^{(i)} + \ell_N\big(x_N^{(i)}; \xi_N^{(i)}\big)$;

  $\mathcal{L} \leftarrow \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}^{(i)}$; // batch DPC loss

  Compute $\nabla_\theta \mathcal{L}$ e.g., BPTT or adjoint sensitivity

  $\theta \leftarrow \mathbb{O}\big(\theta, \nabla_\theta \mathcal{L}\big)$; // e.g., Adam step

return π_θ;

Forward pass

DPC Policy Optimization Algorithm

Algorithm 5: Self-Supervised DPC Policy Optimization

Input: Horizon N, batch size B; initial-state distribution $\mathcal{X}_0$, scenario distribution $\mathcal{D}$; dynamics $x_{k+1} = f(x_k, u_k; \xi_k)$; policy class $u_k = \pi_\theta(x_k; \xi_k)$; costs ℓ, ℓ_N; penalties p_x, p_u with weights Q_x, Q_u; constraints g, h; optimizer $\mathbb{O}$

Output: Trained explicit policy $\pi_{\theta^\star}$

while *not converged* **do**

  Sample mini-batch $\{(x_0^{(i)}, \xi^{(i)})\}_{i=1}^{B} \sim \mathcal{X}_0 \times \mathcal{D}$;

  for $i \leftarrow 1$ **to** B **do**

    for $k \leftarrow 0$ **to** $N-1$ **do**

      $u_k^{(i)} \leftarrow \pi_\theta\big(x_k^{(i)}; \xi_k^{(i)}\big)$;

      $x_{k+1}^{(i)} \leftarrow f\big(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}\big)$;

      $\ell_k^{(i)} \leftarrow \ell\big(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}\big) + Q_x\, p_x\big(g(x_k^{(i)}; \xi_k^{(i)})\big) + Q_u\, p_u\big(h(u_k^{(i)}; \xi_k^{(i)})\big)$;

    $\mathcal{L}^{(i)} \leftarrow \frac{1}{N} \sum_{k=0}^{N-1} \ell_k^{(i)} + \ell_N\big(x_N^{(i)}; \xi_N^{(i)}\big)$;

  $\mathcal{L} \leftarrow \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}^{(i)}$; // batch DPC loss

  Compute $\nabla_\theta \mathcal{L}$ e.g., BPTT or adjoint sensitivity

  $\theta \leftarrow \mathbb{O}\big(\theta, \nabla_\theta \mathcal{L}\big)$; // e.g., Adam step

return π_θ;

Backward pass
Gradient step

DPC Policy Optimization Algorithm

Algorithm 5: Self-Supervised DPC Policy Optimization

Input: Horizon N, batch size B; initial-state distribution $\mathcal{X}_0$, scenario distribution $\mathcal{D}$; dynamics $x_{k+1} = f(x_k, u_k; \xi_k)$; policy class $u_k = \pi_\theta(x_k; \xi_k)$; costs ℓ, ℓ_N; penalties p_x, p_u with weights Q_x, Q_u; constraints g, h; optimizer $\mathbb{O}$

Output: Trained explicit policy $\pi_{\theta^\star}$

while *not converged* **do**

  Sample mini-batch $\{(x_0^{(i)}, \xi^{(i)})\}_{i=1}^{B} \sim \mathcal{X}_0 \times \mathcal{D}$;

  for $i \leftarrow 1$ **to** B **do**

    for $k \leftarrow 0$ **to** $N-1$ **do**

      $u_k^{(i)} \leftarrow \pi_\theta\big(x_k^{(i)}; \xi_k^{(i)}\big)$;

      $x_{k+1}^{(i)} \leftarrow f\big(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}\big)$;

      $\ell_k^{(i)} \leftarrow \ell\big(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}\big) + Q_x\, p_x\big(g(x_k^{(i)}; \xi_k^{(i)})\big) + Q_u\, p_u\big(h(u_k^{(i)}; \xi_k^{(i)})\big)$;

    $\mathcal{L}^{(i)} \leftarrow \frac{1}{N} \sum_{k=0}^{N-1} \ell_k^{(i)} + \ell_N\big(x_N^{(i)}; \xi_N^{(i)}\big)$;

  $\mathcal{L} \leftarrow \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}^{(i)}$; // batch DPC loss

  Compute $\nabla_\theta \mathcal{L}$ e.g., BPTT or adjoint sensitivity

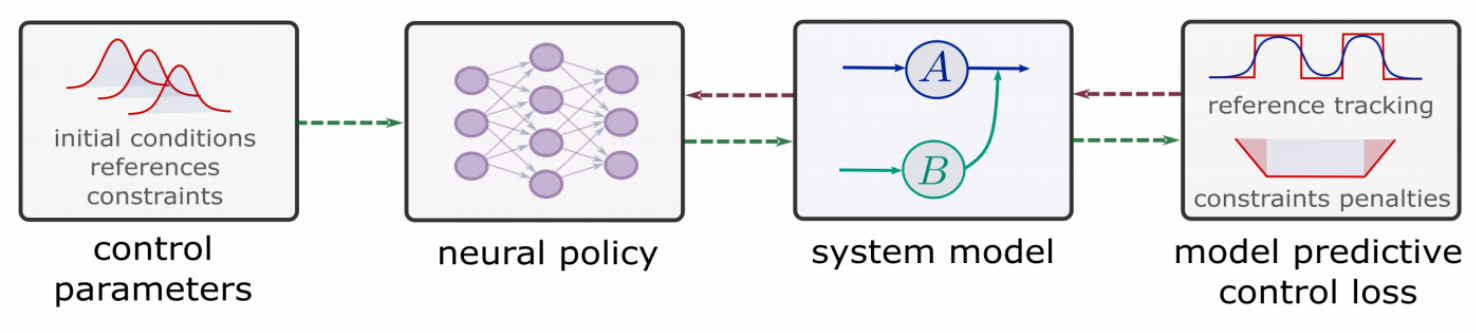
  $\theta \leftarrow \mathbb{O}\big(\theta, \nabla_\theta \mathcal{L}\big)$; // e.g., Adam step

return π_θ;

DPC learns approximate explicit MPC policies through sampling-based self-supervised policy optimization.

No MPC demonstrations required.
No learned value function.
No expensive online optimization.
Offline computation is amortized across a distribution of problems.

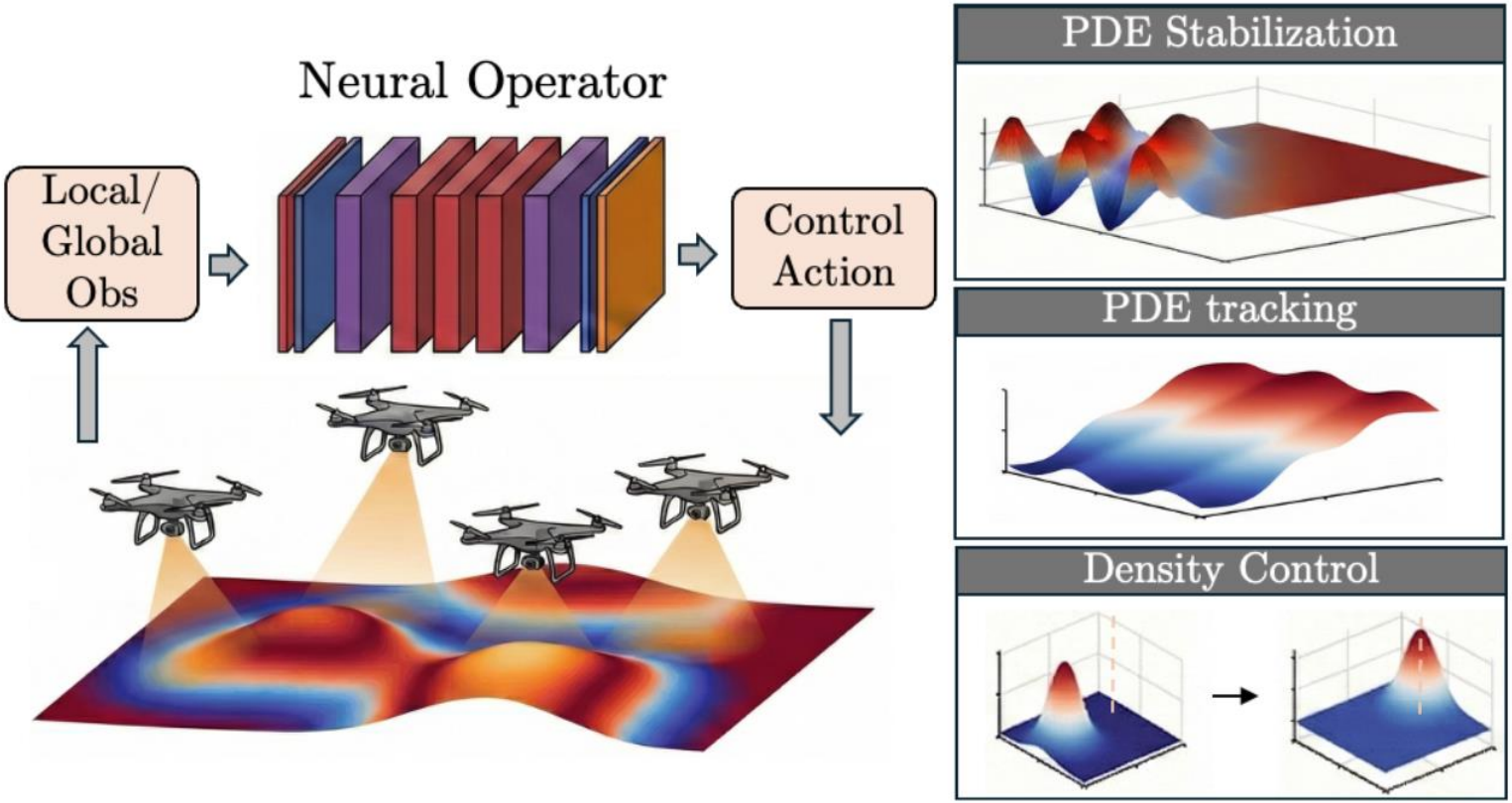
DPC Example in NeuroMANCER SciML Library



https://colab.research.google.com/github/pnnl/neuromancer/blob/master/examples/control/Part_3_ref_tracking_ODE.ipynb

github.com/pnnl/neuromancer

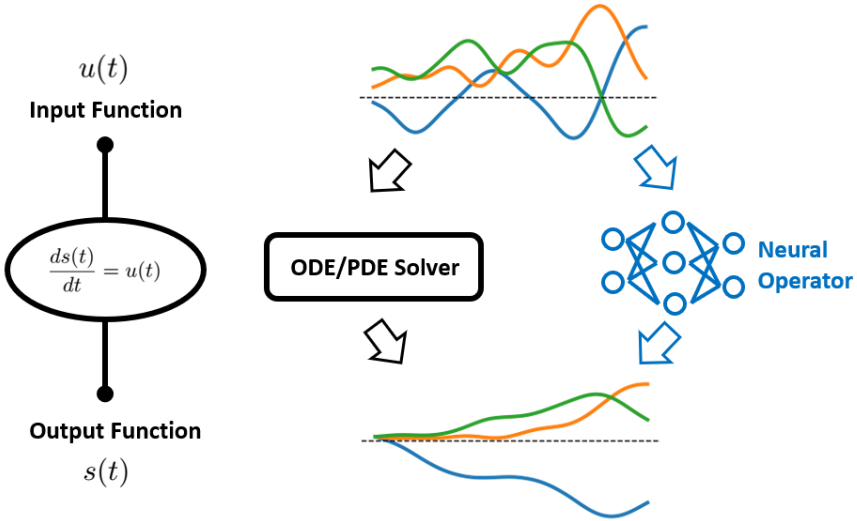
Learning to Control Partial Differential Equations with Neural Operators



P. Zanotta, H. Zheng, D. R. Sharkar, S. Goswami, J. Drgoňa, Cardinality-Invariant Neural Operator Policies for Scalable PDE Control, ICML, 2026, https://solaris-jhu.github.io/CINOC-website/

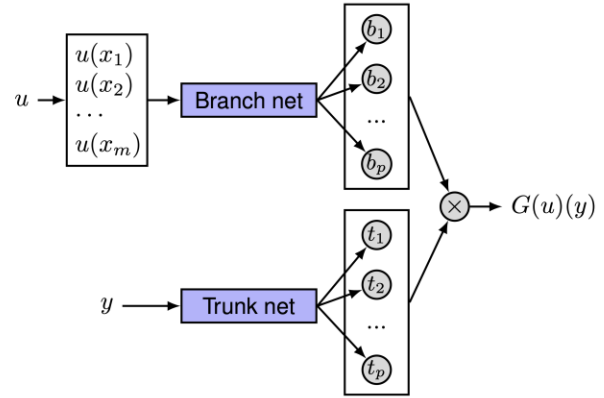
Neural Operators in a Nutshell

Neural networks learn maps between vector spaces.
In contrast, neural operators learn maps between infinite-dimensional function spaces.

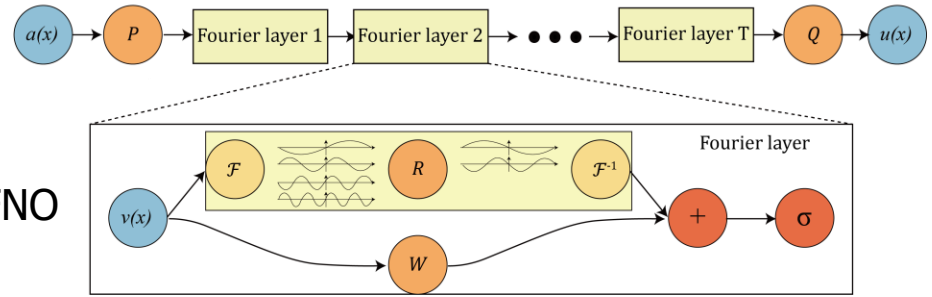


https://towardsdatascience.com/operator-learning-via-physics-informed-deeponet-lets-implement-it-from-scratch-6659f3179887/

DeepOnet

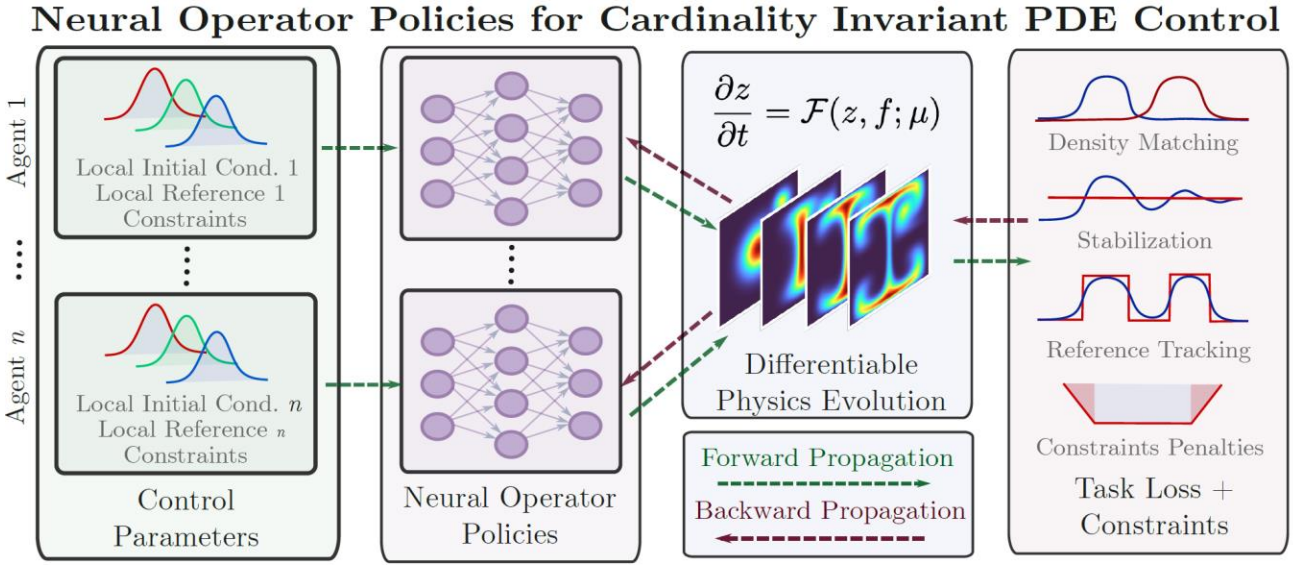


FNO



Lu, Lu, et al. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nature Machine Intelligence, 2021
Nikola Kovachki, et al., Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs, JMLR, 2023

Learning to Control PDE Example in JAX



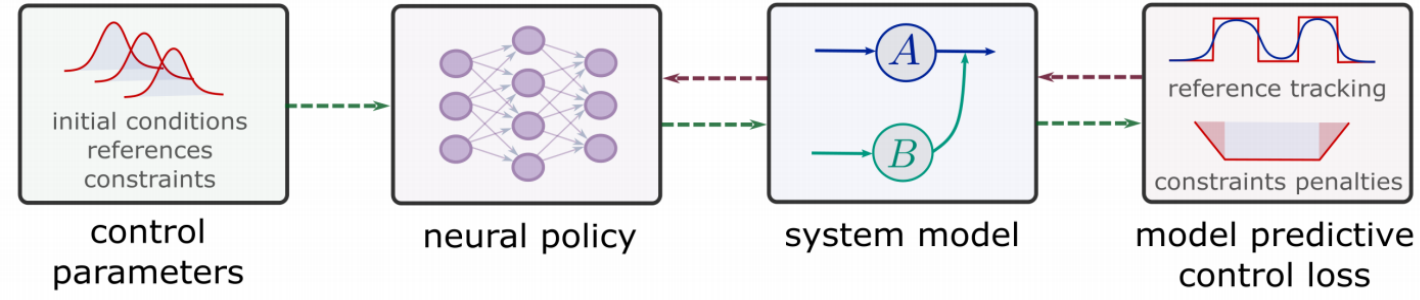
Pietro Zanotta

https://github.com/SOLARIS-JHU/CINOC-website

Learning to Control (L2C) Summary

- **L2C via Differentiable Predictive Control**
  - Learns approximate explicit MPC policies through sampling-based self-supervised optimization.
  - Shifts computational burden offline, enabling real-time deployment via a single policy evaluation.
  - Works with differentiable ODE/PDE solvers and a wide range of learned surrogate models.

- **Challenges**
  - Control of hyperbolic PDEs
  - Handling with stiff and hybrid systems
  - Gradients ill conditioning
  - Offline pre-training vs online adaptation
- **Code**
  - github.com/pnnl/neuromancer
  - github.com/SOLARIS-JHU/CINOC
- **Hiring!** jdrgona1@jh.edu

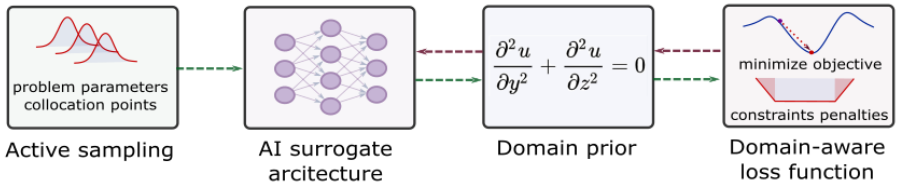


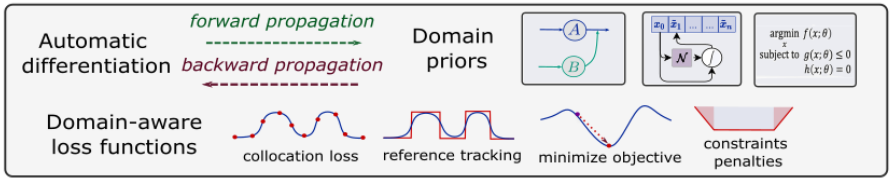
SciML Tutorial Summary

Scientific machine learning (SciML) integrates deep learning, constrained optimization, physics-based modeling, and control in a unified abstraction.

- Learning to optimize (L2O)
- Learning to model (L2M)
- Learning to control (L2C)

sciml-l4dc2026.github.io/SciML-L4DC2026/





Lab websites and contact

Thomas Beckers

- www.tbeckers.com/
- thomas.beckers@vanderbilt.edu

Truong Nghiem

- truong.nxtlab.org/
- truong.nghiem@ucf.edu

Jan Drgona

- solaris-jhu.github.io/solaris-website/
- drgona.github.io/
- jdrgona1@jh.edu

VANDERBILT UNIVERSITY

University of Central Florida

JOHNS HOPKINS UNIVERSITY

L2C Backup Slides

DPC vs Model-based Reinforcement Learning

DPC is related to **MBRL** in that both leverage **model of dynamics**, but DPC utilizes **differentiable** closed-loop models and cost functions, allowing direct policy gradients **without requiring a learned critic**.

Empirical risk minimization problem:

$$\min_{\theta} \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\xi}_k} \left(\sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k, \boldsymbol{\xi}_k) + p_N(\mathbf{x}_N) \right)$$

$$\text{s.t. } \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, \boldsymbol{\xi}_k), \;\; k \in \mathbb{N}_0^{N-1}$$

$$\mathbf{u}_k = \pi_\theta(\mathbf{x}_k, \boldsymbol{\xi}_k)$$

$$h(\mathbf{x}_k, \boldsymbol{\xi}_k) \leq 0$$

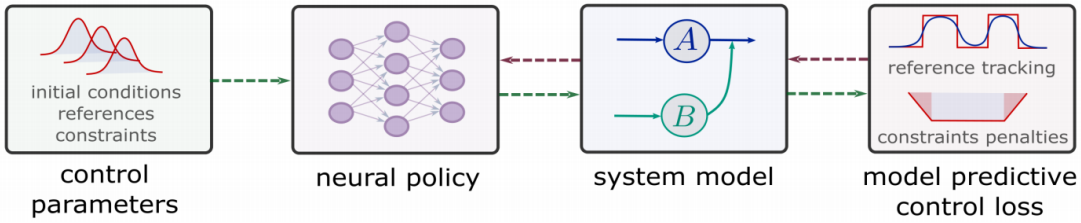
$$g(\mathbf{u}_k, \boldsymbol{\xi}_k) \leq 0$$

Critic parametrized by differentiable MPC loss function:

$$\mathcal{L}_{\text{DPC}} = \frac{1}{mN} \sum_{i=1}^{m} \Big(\sum_{k=0}^{N-1} \big(\ell(\mathbf{x}_k^i, \mathbf{u}_k^i, \boldsymbol{\xi}_k^i) + p_x(h(\mathbf{x}_k^i, \boldsymbol{\xi}_k^i)) + p_u(g(\mathbf{u}_k^i, \boldsymbol{\xi}_k^i)) \big) + p_N(\mathbf{x}_N^i) \Big)$$

Policy gradient via automatic differentiation:

$$\nabla_\theta \mathcal{L}_{\text{DPC}} = \left(\frac{\partial \ell}{\partial \mathbf{x}} + \frac{\partial p_x}{\partial \mathbf{x}} + \frac{\partial p_N}{\partial \mathbf{x}} \right) \cdot \frac{\partial \mathbf{x}}{\partial \mathbf{u}} \cdot \frac{\partial \mathbf{u}}{\partial \theta} + \left(\frac{\partial \ell}{\partial \mathbf{u}} + \frac{\partial p_u}{\partial \mathbf{u}} \right) \cdot \frac{\partial \mathbf{u}}{\partial \theta}$$



$\ell(\cdot)$: stage cost
$p_x(\cdot)$, $p_u(\cdot)$: constraints penalties
$p_N(\cdot)$: terminal cost
$\frac{\partial \mathbf{x}}{\partial \mathbf{u}}$: differentiable dynamics
$\frac{\partial \mathbf{u}}{\partial \theta}$: differentiable policy

J. Drgoňa, A. Tuor and D. Vrabie, *Learning Constrained Parametric Differentiable Predictive Control Policies With Guarantees, in IEEE Transactions on Systems, Man, and Cybernetics: Systems, 2024*

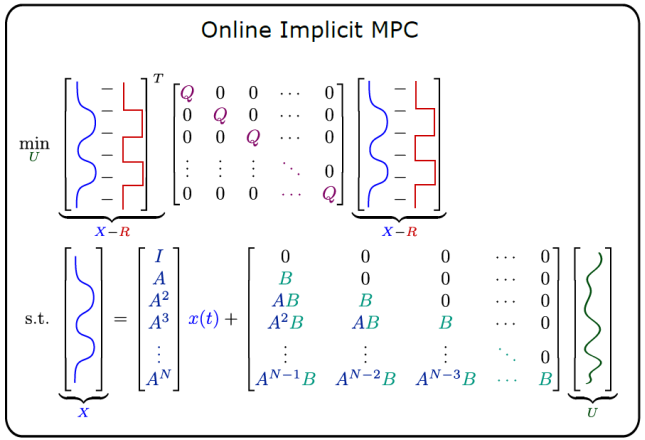
J. Drgoňa, et al., *Differentiable Predictive Control: An MPC Alternative for Unknown Nonlinear Systems using Constrained Deep Learning*, Journal of Process Control, 2022

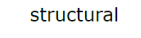
DPC vs Model Predictive Control

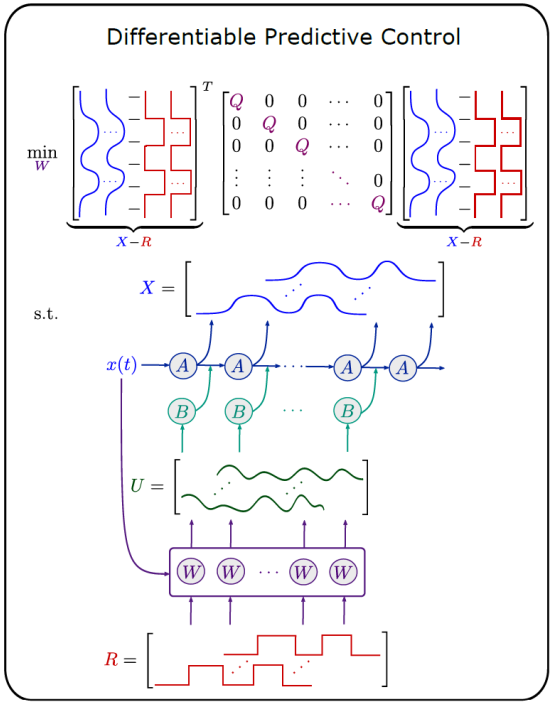
DPC is also related to MPC, but instead of single instance online optimization, the **DPC learns parametric explicit policy offline**, over distribution of parametric instances in batched setting.

There is a **structural equivalence** between *single shooting* formulation of MPC problem and the *unrolled closed-loop system* dynamics in DPC.

DPC is also related to **explicit MPC**, as it solves a **parametric optimal control problem** via gradient-based policy optimization.







Compact notation

system dynamics	$x_{k+1} = f(Ax_k, Bu_k)$
control policy	$U = f_W(x(t), R)$
training dataset	$\{(x(t)^{(1)}, R^{(1)}), \ldots, (x(t)^{(m)}, R^{(m)})\}$

J. Drgoňa, A. Tuor and D. Vrabie, *Learning Constrained Parametric Differentiable Predictive Control Policies With Guarantees, in IEEE Transactions on Systems, Man, and Cybernetics: Systems, 2024*

J. Drgoňa, et al., *Differentiable Predictive Control: An MPC Alternative for Unknown Nonlinear Systems using Constrained Deep Learning*, Journal of Process Control, 2022

DPC Advantages

Advantages

- **Amortized control:** Near-instant policy evaluation replaces online MPC solves. Enables fast high-rate control or embedded deployment where full MPC is too slow. Can provide up to 5 orders of magnitude faster inference time compared to iterative solvers.
- **End-to-end design:** Can co-optimize policy with differentiable system models, estimators, and reference generators.
- **Constraint awareness:** Can integrate differentiable projections, control Lyapunov and control barrier functions, terminal sets, or differentiable MPC blocks.
- **Data efficiency (model-based):** Uses dynamics knowledge and system model rollouts to generate self-supervised training data; does not require labels from optimal solutions.
- **Model flexibility:** Supports a wide range of system model representations, including data-driven representations using NODEs, neural operators, or neural state space models.

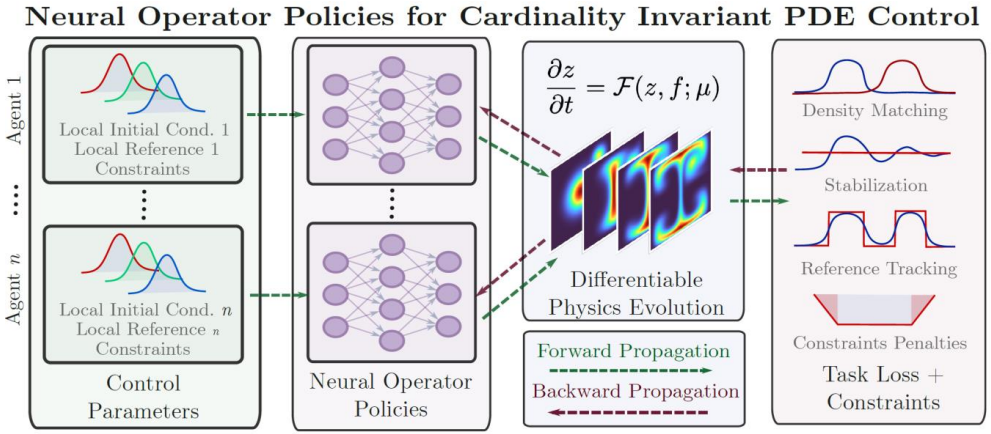
DPC Disadvantages

Pitfalls

- **Model mismatch:** Performance hinges on f; robustness needs domain randomization, noise-injection, or adaptation.
- **Feasibility/stability guarantees:** Require proper architectural choices (terminal ingredients, invariant sets, CLF/CBF shaping, safety filters).
- **Handling stiff systems:** Dealing with stiff dynamical system models is problematic due to the numerical instabilities and exploding/vanishing gradient problem.
- **Training pathologies:** BPTT through long horizons can be unstable; loss balancing between tracking and constraints is delicate.
- **Coverage/generalization:** Policies may fail under rare events or OOD conditions without distributional robustness.

PDE Control as Self-Supervised Neural Operator Learning

We learn the multi-agent policy operator using differentiable predictive control algorithm with differentiable PDE solvers.



Algorithm 1 Self-Supervised Operator Policy Training

Input: distribution $\mathcal{D}_{\text{prob}}$, horizon T, number of agents M
Initialize: shared policy parameters $\boldsymbol{\theta}$
while not converged **do**
  {**1. Data Sampling**}
  Sample instances $\boldsymbol{\psi} = (z_0, z_{\text{target}}, \dots) \sim \mathcal{D}_{\text{prob}}$
  Initialize state z_0, actuator positions $\{\boldsymbol{\xi}_{i,t=0}\}_{i=1}^{M}$
  for $t = 0$ to T **do**
    {**2. Shared Policy Inference**}
    for $i = 1$ to M **do**
      $z_i^{\text{patch}} \leftarrow \mathcal{O}bs(z_t, \boldsymbol{\xi}_{i,t}, \boldsymbol{\psi})$
      $[u_i(t), \boldsymbol{v}_i]^\top \leftarrow \mathcal{G}_{\boldsymbol{\theta}}(z_i^{\text{patch}}, \boldsymbol{\xi}_{i,t})$
    end for
    {**3. Differentiable Physics Step**}
    $z_{t+1}, \{\boldsymbol{\xi}_{t+1}\} \leftarrow \Psi(z_t, \{\boldsymbol{\xi}_t\}, \boldsymbol{u}_t^M)$
    {**4. Loss**}
    $\mathcal{J} \leftarrow \mathcal{J} + \ell(z_{t+1}, \boldsymbol{u}_t^M, \boldsymbol{\psi})$
  end for
  Update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{J}$ {Backprop via Physics}
end while

P. Zanotta, H. Zheng, D. R. Sharkar, S. Goswami, J. Drgoňa, Cardinality-Invariant Neural Operator Policies for Scalable PDE Control, ICML, 2026, https://solaris-jhu.github.io/CINOC-website/